



Equation Discovery for Classification: Finding Interpretable Symbolic Specifications of the Decision Boundary

Guus Toussaint¹ · Arno Knobbe¹

Received: 28 February 2026 / Revised: 19 August 2026 / Accepted: 25 August 2026
© The Author(s) 2026

Abstract

Equation discovery has traditionally focused on regression, where the goal is to recover analytical expressions that model numerical targets. In this paper, we extend this paradigm to binary classification and introduce Equation Discovery for Classification (EDC), a framework that discovers concise symbolic expressions that explicitly define decision boundaries. EDC searches over a configurable grammar of analytical expressions using beam search and optimises equation parameters via dedicated numerical procedures, yielding a single interpretable equation whose sign determines class membership. We design a redundancy-aware grammar that balances expressivity and tractability, enabling the discovery of non-linear decision boundaries while maintaining interpretability. Through experiments on artificial datasets with known generating mechanisms, we show that EDC reliably reconstructs complex target boundaries, including XOR-like and interaction-driven structures, and adapts effectively under increasing levels of noise. Notably, in noisy settings EDC can outperform the original generating equation by approximating the implicit, noise-shifted decision boundary. On UCI benchmark datasets, EDC consistently outperforms existing symbolic classification approaches and other interpretable baselines, while achieving performance competitive with state-of-the-art black-box models. Although computationally more demanding than standard classifiers, we demonstrate that substantial speed-ups can be achieved with reduced search depth and simplified grammars at minimal loss of predictive performance. Overall, EDC provides a principled bridge between symbolic regression and classification, offering a transparent yet expressive alternative to black-box models for applications where interpretability of the decision boundary is essential.

Keywords Machine learning · Binary classification · Equation discovery · Explainable machine learning · Symbolic classification

Editors: Gianvito Pio, Jurica Levatić, Nikola Simidjievski.

✉ Guus Toussaint
g.toussaint@liacs.leidenuniv.nl
Arno Knobbe
a.j.knobbe@liacs.leidenuniv.nl

¹ Leiden University, Leiden, The Netherlands

1 Introduction

Equation discovery, the task of discovering analytical functions that model data, is well known for its use in regression settings, where it is then often referred to as Symbolic Regression (Augusto & Barbosa, 2000; Billard & Diday, 2002). Many real-world applications require a model to be interpretable, particularly in scenarios where a decision could have substantial consequences. Therefore, there is a movement in machine learning that tries to build models that can be easily interpreted and, where needed, corrected by the practitioners dealing with these models. The framework of equation discovery fits this trend perfectly. The result generated does not consist of a large, opaque model that calculates the value for the new data points but rather a simple equation that describes the nature of the data. Furthermore, a domain expert can adjust the pattern language of equations considered by the algorithm, thus allowing the algorithm to adapt to different scenarios where, for example, a non-linear translation of the data makes sense.

As mentioned, the focus of equation discovery (ED) has largely been on regression problems. In this work, we use the ED framework to tackle problems in the classification domain (Toussaint & Knobbe, 2025). The central idea is to generate concise and readable equations that define the decision boundary between classes, which domain experts can then evaluate. We show that a classification-oriented ED algorithm may perhaps not beat the state of the art of well-balanced algorithms such as Random Forests or Multi-Layer Perceptrons but be in the same ballpark in terms of classification performance and certainly produce more transparent models than, e.g. Random Forests would.

When used in a regression setting, the equations considered are candidate functions to model the (numeric) target as a function of several input attributes. These equations typically involve arithmetic operators as well as a set of standard analytical functions (log, exp, sin,...). Such an approach could then be used to automate scientific discovery (such as Boyle's law, which relates the pressure of a gas in a vessel to its volume and temperature (Levine, 1978)). However, the approach can also be used in more noisy statistical settings, where a regression model captures the influence of certain parameters on the target value. In broad strokes, ED has two separate challenges: first, finding the structure of the equation, and second, optimising the parameters of the candidate equation relative to the available data. Many standard optimisation algorithms are available for the latter, e.g., Gradient Descent (Boyd & Vandenberghe, 2004). ED then primarily becomes a discrete optimisation problem defined by a search space of allowable equations and a strategy to traverse this search space, either exhaustively or heuristically. Like (Todorovski & Dzeroski, 1997), we define the pattern language, and hence the search space, by means of a configurable grammar that produces a language of equations, typically up to a certain complexity. We provide an example of a grammar that works quite well on datasets involved in our experiments, but we would like to stress that the exact grammar is a parameter of the proposed method. A knowledgeable domain expert may add certain functions to the grammar in order to enrich or reformulate the classification problem in terms of the functions provided.

We model the decision boundary of the classifier in the following basic manner: $T = \text{true}$ iff $f(x) \geq \theta$, where $f(x)$ is the equation discovered by ED and θ is a threshold. θ is found during induction by computing the ROC curve of potential thresholds and then picking the θ that optimises the accuracy of the classifier (i.e. assuming equal miss-classification costs). In other words, the function f defines a landscape in terms of the available

attributes, and whenever the landscape is above some threshold θ , the classification is true. It is important to note that this paper focuses on *binary classification*; extensions to multi-class classifications are possible but outside the scope of this paper. For simple formulae, such as linear equations (hyperplanes), this setting is equivalent to existing methods such as logistic regression or SVMs with a linear kernel (Flach, 2012; Hosmer & Lemeshow, 2000). However, ED comes into its own when richer expressions are involved.

The contributions of our paper can be summarised as follows:

- We propose a method called Equation Discovery for Classification (EDC), that is both interpretable and, with the correct building blocks for the domain at hand, is able to draw complex decision boundaries such that the performance is comparable to the current state of the art in classification.
- With the aim of avoiding redundancy in the equation search space, we suggest a specifically-designed grammar that works in generic cases, and can be refined with application-specific constructs.
- We demonstrate that the algorithm is able to reconstruct a hard-coded decision boundary in artificial data and is able to model challenging XOR-like problems.
- We demonstrate the performance of our model on UCI datasets, where we compare it to state-of-the-art algorithms in terms of area under the curve (AUC), interpretability and model size.

Along with this paper, we publish the code¹ and experimental setup^{2,3} so that the work presented here can be completely reproduced. This paper forms a substantial extension of our prior work on symbolic classification “*EDC: Equation Discovery for Classification*”, Toussaint & Knobbe (2025) through extended experimentation, revealing EDC’s behaviour under varying levels of noise and over different degrees of symbolic search.

2 Related Work

The core idea of equation discovery, as it has been presented, is to find a function that fits the data. To achieve this, a search strategy has to be defined; previous works broadly identify two types of search strategies: exhaustive and heuristic search. Exhaustive search focuses on defining a manageable search space so that all possible equations within that search space can be evaluated. One common approach to exhaustive search for equation discovery is to use a context-free grammar (Todorovski & Dzeroski, 1997). Implementing a heuristic search algorithm that traverses this search space more efficiently eliminates the need for restricting the initial search space. Genetic programming has been successfully implemented as a heuristic approach in equation discovery (Antonov et al., 2024; Augusto & Barbosa, 2000; Haeri et al., 2017). Heuristic search algorithms for equation discovery come in two flavours. Some try to optimise the structure and the parameters of the equation

¹The code of the core algorithm can be found in the following repository: <https://github.com/GuusToussaint/EDC-core>.

²The code for reproducing the experiments can be found at the following link <https://github.com/GuusToussaint/EDC-experiments>.

³The datasets can be found at the following link: <https://anonymous.4open.science/r/EDC-datasets-B9F7/>.

simultaneously, while others first try to find the structure, and optimise the parameters at a later stage.

Regression is the dominant data mining task in equation discovery (in which case people speak of *symbolic regression*). The aim here is to discover an analytical expression involving various arithmetic operations and closed-form functions (such as *log*, *sin*, etc.) that model a supervised dataset. The target expression typically exhibits a tree-shaped structure (which needs to be discovered) and involves one or more parameters (that need to be ‘fit’ to the data). For a complete overview of the symbolic regression approach, we refer to the survey paper by Makke and Chawla (2024).

In recent years, several researchers attempted to apply the machinery of symbolic regression to the classification task, sometimes called Symbolic Classification (SC). Schwab and Link (2011) introduced the topic by describing a simple yet elegant alteration of the classic symbolic regression approach using genetic programming that can solve binary classification problems. The algorithm, often referred as AMAXSC, produces a symbolic expression using Genetic Programming, which is then thresholded to produce a discrete decision boundary (much like in our setting). While AMAXSC produces interpretable equations, the experiments show that the method struggles to discover concise and accurate models, often choosing to improve intermediate expressions by adding additional terms, rather than extensively optimising the model’s parameters. This tendency can be mitigated by severely limiting the allowed operators, variables and functions to occur in the expressions.

To improve upon the initial design of AMAXSC and extend it to a multi-label classification setting, Ingalalli et al. (2014) introduced the Multi-dimensional Multi-class Genetic Programming algorithm (M₂GP). They argue that a single expression is not informative enough to solve multi-class classification problems. Therefore, they use a multi-expression representation, where each sample is mapped to a custom number of dimensions (d), where each dimension is a learned equation. Thus, this approach can be seen as mapping any sample to a latent feature space using d transformations. The final classification is done by calculating the distance between the sample and the centroid of each class in the latent feature space. The initial experiments showed promising results with performance comparable to state-of-the-art classification methods. However, as a result of having (an unspecified number of d) latent features, interpretability is compromised (which is understandable, given the challenges of multi-class classification).

The original method has since been improved by eliminating the need to manually select the number of dimensions d and instead learning it as a parameter within the method. M3GP introduced by Muñoz et al. (2015) extends M₂GP by allowing the search process to progressively search for the optimal number of new dimensions that maximise the classification accuracy. In the most recent development, La Cava et al. (2019) introduced M4GP, which improves upon M₂GP and M3GP by simplifying the program encoding, using advanced selection methods, and archiving solutions during the run. In our experiments, we demonstrate that M4GP is not able to reach state-of-the-art performance on *binary* classification in terms of model accuracy.

Korns (2018) have developed the Multilayer Discriminant Classification (MDC) algorithm, a computationally efficient alternative to the M₂GP. Cava et al. (2019) also introduced FEAT, another alternative to M₂GP where the clustering approach is replaced with a linear regression approach. However, the same performance and interpretability drawbacks remain. Our approach aims to improve the interpretation and ease of use in binary classifica-

tion by having only a single expression, while achieving competitive accuracy comparable to the state-of-the-art in classification.

In this work, we also compare our approach to two symbolic regression implementations, EGG-SR as described by Jiang et al. (2026) (EGGP), and PySR by Cranmer (2023). While both of these methods are introduced as symbolic regression algorithms, they can be adapted to the binary classification problem by employing the same thresholding approach mentioned previously.

3 Background

Assume that we have a dataset X consisting of numeric attributes. A data point $x \in X$ consists of d numeric features such that $x \in \mathcal{R}^d$. When faced with a dataset that contains categorical features, one-hot encoding transforms them into a set of numeric 0,1-features. As this may explode the number of features when dealing with high-cardinality categorical features, we take the pragmatic approach to group one-hot encoded features whose frequency of occurrence is smaller than or equal to 2%.

To evaluate the fitness of an equation, we use the Log Loss (Cybenko et al., 1999), also known as the logistic loss or the cross-entropy loss. Since the input for the Log Loss function represents probabilities, only values between 0 and 1 are allowed. Therefore, the output of the equation is passed through the logistic function before calculating the loss.

Once an equation is found, the results are ranked based on their probability scores. This ranking is used to calculate the area under the receiver operating characteristic curve (AUC), which determines the final performance of the model. The equation does not assign direct labels to input data but rather a probability score (Flach, 2012). An appropriate threshold can be chosen based on the costs of mislabelled false cases as true and vice versa. In this work, we will choose a threshold θ that maximises the accuracy (misclassification costs are assumed to be equal).

4 Method

The proposed framework for equation discovery in a classification setting consists of two main components. First, candidate equations are constructed during the *search* step. Second, the constants in these candidate equations are optimised during the *optimisation* step.

4.1 Search

We will define the extent of the search space by means of a context-free grammar, further defined below. However, it is safe to say that, in most cases, the defined search space will be too large to examine exhaustively. Even when the search depth is limited, with the growing number of attributes in the dataset, the number of possible equations will explode quickly. For this reason, we will adopt a heuristic search strategy based on *beam search*, which provides an attractive balance between exploration and exploitation (Kumar et al., 2013).

The notion of using a context-free grammar to define the search space of possible equations in equation discovery was first introduced by Todorovski and Dzeroski (1997). The

overall search space is limited by declaring a grammar that describes all possible valid equations. However, the grammar can be defined so that all relevant equations remain in the constrained search space. Ideally, a domain expert defines the grammar for each domain the algorithm applies to. A domain expert can provide information on translation combinations of relevant features for the domain.

Our generic, application-independent grammar should include expressive and effective equations that are at the same time interpretable. It includes equations that consist of basic arithmetic operations, including addition, multiplication, and exponentiation. It is designed with an eye on limiting the level of redundancy in the search space such that two equations that are syntactically different but semantically equivalent are only considered once. As a simple example, $x + c \cdot y$ is equivalent to $x - c \cdot y$, since subtraction can be achieved by negating c , so the latter is not part of the grammar. Other opportunities for pruning the search space are explained below the definition of the grammar.

The grammar is defined as follows: $G = (\mathcal{N}, \mathcal{T}, \mathcal{R}, \mathcal{S})$, which contains the following sets of symbols and operations:

- $\mathcal{N}$ contains all non-terminal symbols.
- $\mathcal{T}$ contains all terminal symbols.
- $\mathcal{R}$ contains the rewrite rules in the form $A \rightarrow \alpha$ where $A \in \mathcal{N}$ and $\alpha \in (\mathcal{N} \cup \mathcal{T})^*$.
- $\mathcal{S}$ contains the start symbol.

For our experiments, we have constructed the following grammar G_s :

- $\mathcal{N}_s = \{V, B, X\}$
- $\mathcal{T}_s = \{x_1, x_2, c, +, \cdot, \exp\}$
- $\mathcal{R}_s = \{V \rightarrow c \mid V + B, B \rightarrow c \cdot X \mid c \cdot \exp(c \cdot X) \mid c \cdot X \cdot X, X \rightarrow x_1 \mid x_2\}$
- $\mathcal{S}_s = \{V\}$.

Note that this grammar only applies to datasets containing two features, i.e., x_1 and x_2 ; for our experiments, the non-terminal set X is expanded to fit the number of features available in the dataset.

When dealing with real-world data, the numerical inputs might have an arbitrary scale, which affects the range of the constants to optimise, something we would like to avoid. For this reason, prior to actually starting the EDC algorithm, we normalise the data by means of linear scaling to the interval $[-1, 1]$. We note that when the data contains extreme values, other normalisation techniques, e.g. *z-normalisation*, might be more suitable.

Our choice of grammar is based on a number of design principles. First off, to keep the search space to a manageable size, we require our equation to be a set of summands: a sequence of building blocks that are connected through addition. As an additional benefit, this keeps derived equations similar to their ancestors: the derived equation just has a summand added and remains unchanged otherwise. Furthermore, we only use addition, not subtraction, between summands because each summand has a constant c that the numeric optimiser can set to a negative number if necessary. As for the summands, we allow a linear term x , multiplication of two features $x \cdot y$ (note that this includes quadratic terms x^2), and exponentiation $\exp(x)$.

For each such term, one or more constants c_i are introduced in specific locations to further parameterise the equation. For the $\exp$ function, we add a constant with multiplication ‘inside’ since that cannot be rewritten into a simpler form. Note that addition of a constant inside $\exp$ is superfluous because that can be rewritten as follows: $\exp(a + x) = \exp(a) \cdot \exp(x) = c \cdot \exp(x)$

The grammar is not set in stone; thus, when deploying the proposed classifier in a domain with different characteristics, it is possible to design a different grammar with the required characteristics.

While the equation language Λ , and thus the search space, is defined by the context-free grammar defined above, we use *beam search* to heuristically traverse a relevant but limited portion of this search space. This search algorithm has been successfully applied in many mining settings, e.g., subgroup discovery (Meeng et al., 2014) or multi-label learning (Kumar et al., 2013). Figure 1 illustrates an example of beam search.

Our implementation builds equations within Λ by iteratively adding summands to the equation, starting from a starting constant c_0 . These summands are essentially options of the rewrite rule for B : $c \cdot X$, $c \cdot \exp(c \cdot X)$, and $c \cdot X \cdot X$. At each level, a single summand is added. A summand may have one or two constants c , so that a candidate equation at depth d may have up to $2d + 1$ constants. For example, the equation $c_0 + c_1 \cdot x_1 + c_2 \cdot \exp(c_3 \cdot x_2)$ is found at search depth $d = 2$ by refining the starting equation c_0 twice, as follows:

$$\begin{aligned} &c_0 \\ &c_0 + c_1 \cdot x_1 \\ &c_0 + c_1 \cdot x_1 + c_2 \cdot \exp(c_3 \cdot x_2). \end{aligned}$$

This equation has three summands, but four constants.

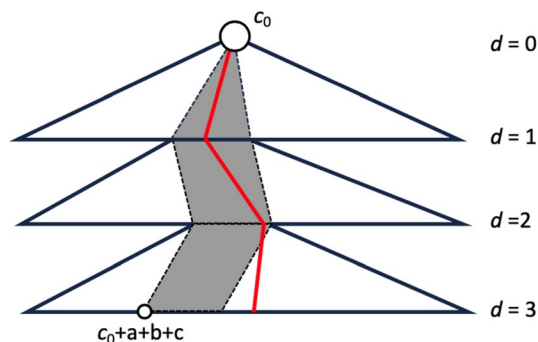


Fig. 1 Beam search for equation discovery. The figure shows four search levels, starting at the top with the root equation consisting of a single constant c_0 . The refinement operator produces a collection of candidate equations indicated by the top triangle. The w most promising equations of these (the beam) are then refined to the next level, and so on. The white triangles combined indicate the collection of candidates considered. The grey beam indicates the candidates selected for refinement. The total search space grows exponentially in d and is much larger than the triangles indicated here. The red line shows the alternative best-first search, which may produce a less optimal result due to its greedier nature

4.2 Optimisation

Each equation encountered in the search space contains constants that need to be optimised for the target at hand. Although all equations within our grammar are differentiable, initial experimentation with gradient descent optimisation proved that such an approach provides poor results, mainly in the presence of $\exp()$ (which is often a considerable fraction of the search space). Our optimizer of choice is therefore the *Hill Climber*, which allocates a portion f of its budget to sample random configurations. Subsequently, the top k configurations are chosen as the starting points for the hill climber. The remaining budget of $n \cdot (1 - f)$ evaluation is then evenly distributed among these top k configurations, for $n \cdot (1 - f)/2km$ evaluations each (where m is the number of features). Each evaluation involves taking a step of size α in both directions for each feature, i.e., moving up and down. These potential steps are then evaluated, and the one that results in the most significant decrease in loss is selected as the next step.

5 Experiments

This section describes the experiments conducted to evaluate the EDC algorithm and its components. The experiments are divided into three parts. First, we identify the best optimiser for optimising the constants. Second, we evaluate the performance of the EDC algorithm under various conditions using a series of artificial datasets. Third, the EDC algorithm is evaluated on a set of UCI datasets and compared to common other classifiers.

5.1 Optimisation

The optimisation of the constants in candidate equations is an important part of our approach; our optimiser must be able to identify a set of constants that minimises the loss function for the proposed equation. We compare the hill climber with a number of off-the-shelf optimisers through the Python SciPy package: Powell, Cobyqa, Cobyla, Nelder-Mead, and Stochastic Gradient Descent. Additionally, we test the Random Sample method as a baseline, which simply uses the entire optimisation budget to test random assignments of the constants, and returns the optimal configuration.

We have designed the following experiment to evaluate the optimisation algorithm's performance on artificial data. From the search space Λ , as defined in Sect. 4, a random equation structure is sampled with either two or three summands. The equation is then completed by sampling random values for each constant from a uniform distribution with bounds $[-10, 10]$. Subsequently, for each equation, a two-dimensional dataset is created by drawing uniformly from the space $[-10, 10] \times [-10, 10]$, where the target values are set according to the decision boundary of the generated equation. If the target is overly unbalanced, i.e. whenever the majority class is larger than 70%, the dataset is recomputed by resampling the equation and associated dataset. Finally, the optimisation algorithm is provided with the equation structure without the values for the constants, as well as the sampled dataset. Note that this approach isolates the optimisation problem from the search for equation structures (which is evaluated in further experiments). This process is repeated 100 times, to test 100 random equations and the mean AUC of the different optimisation algorithms is reported.

Multiple optimisers will be compared, and to make comparisons fair, we provide each algorithm with a budget of $n = 1000$ function evaluations. Some of the optimisers come with hyperparameters. For instance, the gradient descent algorithm has a learning rate, while the hill climber has a step size, a fraction f of random configurations, and a number of configurations k . We employ the hyperparameter optimisation algorithm SMAC3, outlined in Lindauer et al. (2022), utilising a budget of 500 configurations to determine the most suitable values for our specific case. This resulted in a learning rate of $\gamma = 0.05$ for the gradient descent optimisers. For the hill climber, hyperparameter tuning resulted in a fraction of random samples of $f = 0.83$, a step size of $\alpha = 0.29$, and number of configurations $k = 2$.

Table 1 shows the results of our optimisation experiments, in descending order of mean AUC. We observe that off-the-shelf optimisation methods Powell, Cobyqa, Cobyly, Nelder-Mead, and Stochastic Gradient Descent all have an average AUC lower than the Random Sample baseline. The Random Sample baseline achieving an average AUC of 0.9974 indicates that, with a budget of 1000, the optimisation problem can effectively be solved by random sampling. Note, however, that not all algorithms fully utilised the maximum number of 1000 function evaluations. Additionally, the Cobyqa and Cobyly algorithms demonstrated superior runtime performance. However, because optimising the constants is essential for effectively guiding the heuristic search, we selected the best-scoring Hill Climber algorithm for all subsequent experiments.

5.2 Artificial Classification Problems

Now that the best optimiser has been selected and constants can be found efficiently and accurately, we continue to test the actual EDC algorithm, beginning with more artificial datasets. To evaluate the performance of our proposed algorithm in a range of different settings, we have designed two experiments involving artificial data.

For EDC runs, we used the following default settings for its parameters: a maximum search depth of $d = 6$ and a beam width of $w = 10$. The standard grammar as described in Sect. 4.1 was employed. In Sect. 5.3.1, we experiment with different settings.

5.2.1 Equation-Based Artificial Data

First, we want to assess how the EDC algorithm performs under different levels of noise. To accomplish this, we generated artificial datasets with different degrees of noise using the

Table 1 Results of the optimisation experiments, showing the mean AUC (and standard deviation in parentheses) of the optimisation algorithms for a set of 100 randomly sampled equations

Algorithm	AUC	Evaluations	Elapsed time (sec)
Hill Climber	0.9989 (± 0.00)	852 (± 10)	3.4 (± 0.4)
Random Sample	0.9977 (± 0.00)	1000 (± 0)	3.5 (± 0.1)
Cobyly	0.9732 (± 0.14)	65 (± 65)	0.4 (± 0.4)
Cobyqa	0.9721 (± 0.14)	92 (± 64)	0.6 (± 0.4)
Powell	0.9575 (± 0.16)	441 (± 478)	1.5 (± 1.6)
Nelder-Mead	0.8877 (± 0.28)	135 (± 94)	0.5 (± 0.3)
Stochastic gradient descent	0.8061 (± 0.35)	967 (± 129)	6.3 (± 1.4)

Evaluations refers to the average number of function evaluations per equation, and *Elapsed time* reports on the average run time per equation, clearly governed by the number of evaluations

same setup as described in Sect. 5.1. However, after the decision boundary is sampled from the search space, noise is added to the output, making the decision boundary ‘fuzzy’. Data points, after label assignment, are randomly perturbed using Gaussian noise according to the following distribution $\mathcal{N}(0, \sigma^2)$. We vary σ from 0 (no noise) to 8 in increments of 0.5, generating a total of 1,700 artificial datasets. Recall that each feature of our dataset is sampled from $[-10, 10]$, so varying σ between 0 and 8 perturbs the data points from negligible to substantial. Using these datasets, we compare the performance of our EDC algorithm across different noise levels with that of the original decision boundary.

Adding increasing levels of noise to a dataset may push data points near the decision boundary *over* that boundary. Especially with non-linear boundaries and substantial noise, it can be expected that the effective decision boundary will shift from its original location. This is the case because any concavities in the decision boundary will collect more points from across the border than it loses (noisy data points converge on the concave parts). Our experiment allows for an analysis of this interesting phenomenon, by comparing the achieved results of the original decisions boundary (defined by the randomly selected starting equation), to that of EDC (which should approximate the *effective* decision boundary that should start to stray from the original, the more noise is added).

Not only will the effective decision boundary move, it will also change in shape. As more noise is added, any non-linearities will tend to be smoothed out. From our EDC perspective, we thus expect more complex building blocks (e.g., interaction terms, quadratic terms and exponentials) to work better than linear building blocks, but as the level of noise increases, the difference between these two should diminish. In our experiment, we determine the AUC under increasing levels of noise for the full grammar, as well as for a restricted (and thus more computationally attractive) linear grammar.

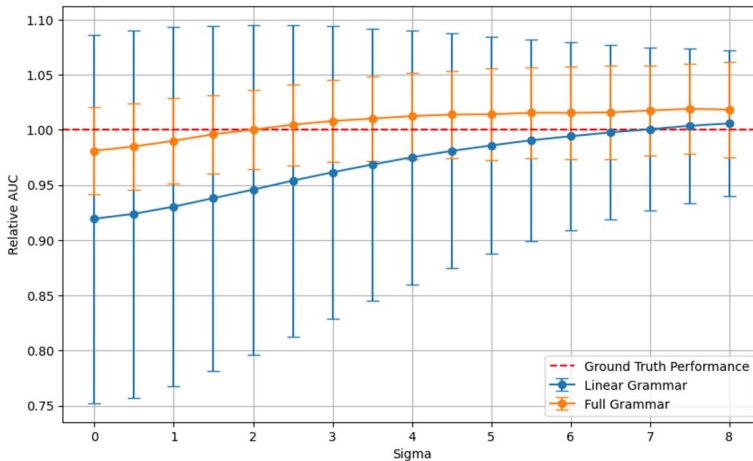
5.2.2 Cluster-Based Artificial Data

A second experiment with artificial datasets is concerned with Gaussian clusters. In this scenario, we create 100 datasets using a mixture of six randomly generated Gaussians. The labels are assigned by randomly assigning two Gaussians to the *true* class and the remaining four to the *false* class. No explicit target equation exists, since the Gaussian intersection determines the dataset’s decision boundary. This experiment is designed to evaluate the performance of the EDC algorithm in a setting where the generated data does not come directly from an equation. This setting is closest to the real world of all the artificial experiments presented in this paper.

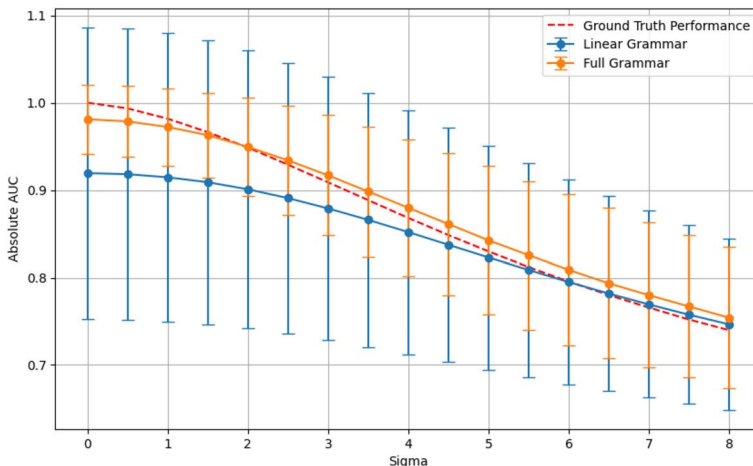
We compare the results on the 100 generated datasets with the following algorithms. First, we compare them against the competing symbolic classification algorithms AMAXSC, M4GP, PySR, and EGGP. Second, we compare them against other interpretable classification algorithms, namely the decision tree (Tree) and the LDA algorithm. Finally, we also compare against the state of the art in terms of classification performance, namely the multi-layer perceptron (MLP), the random forest algorithm (RF), and the support vector machine (SVM) with a radial basis function kernel. The results are presented in terms of the mean AUC with the standard deviation on the 100 generated datasets. We used default settings as published by the authors for all comparison algorithms, without further hyperparameter optimisation.

5.2.3 Results

The results of our experiments with different noise levels are presented in Fig. 2. First of all, we can observe that the performance of EDC using the full grammar is very closely aligned with the performance of the original decision boundary in terms of mean AUC. This indicates that the EDC algorithm is capable of identifying a suitable decision boundary (even sometimes superior) under various levels of noise. Second, we observe that the EDC run with only linear terms also performs well. This indicates that the algorithm is able to



(a)



(b)

Fig. 2 Results of the noise experiment (relative to the ground truth performance (a), and in an absolute sense (b)). The dots indicate the mean score obtained from the experiments, and the error bars indicate the standard deviation. The plots shows the EDC algorithm outperforming the original decision boundary beyond noise level $\mathcal{N}(0, 2)$. At noise level $\mathcal{N}(0, 7)$, the EDC algorithm with only linear terms starts to outperform the original decision boundary

approximate a decision boundary despite this boundary being non-linear. This is relevant since in many real-world problems the actual terms driving the decision boundary may not be known and thus the algorithm must approximate these terms.

Figure 2 shows that at low noise levels, the original decision boundary outperforms both proposed EDC settings. This is to be expected, since the original decision boundary is part of the data-generating process. However, interestingly, we notice that for $\sigma \geq 2$, the EDC run with full grammar outperforms the original decision boundary. This shows that the decision boundary moves as more Gaussian noise is introduced. Furthermore, we observe that beyond $\sigma = 7$, the run with only linear components also starts to outperform the original decision boundary. This shows that as noise levels are increased, the shape of the new decision boundary approaches a linear shape.

Figures 3 and 4 show examples of cases where the EDC algorithm outperforms the original decision boundary. Figure 4 also illustrates that adding noise to the original non-linear decision boundary results in a decision boundary that is more linear.

Figure 5 shows the proposed decision boundary for a single Gaussian clusters dataset. We observe that the EDC algorithm is capable of defining a decision boundary that fits the data, even when the data-generating process is not guided by a *target* decision boundary. The discovered equation is as follows:

$$-0.0439 + 0.0062x_0 + 0.2112x_1 - 0.0496x_0x_1 - 0.66 \exp(-0.28x_0) \quad (1)$$

To get a better understanding of the properties of this decision boundary, it is rewritten as follows:

$$-0.05 + 0.26x_1 + (-0.62x_0 - 0.61)(0.08x_1 - 0.01) - 0.66 \exp(-0.28x_0) \quad (2)$$

Interestingly, the equation defines three regions, where the positive regions of the input space are formed by two convex, non-contiguous regions, separated by a single negative, concave region. This interesting shape is produced by the second building block, which

Fig. 3 This figure shows the target decision boundary (dashed red line), and the decision boundary found by the EDC algorithm (solid blue line). Noise is added to the data, and as a result, the target achieves a lower AUC (0.952) compared to the EDC algorithm (0.978)

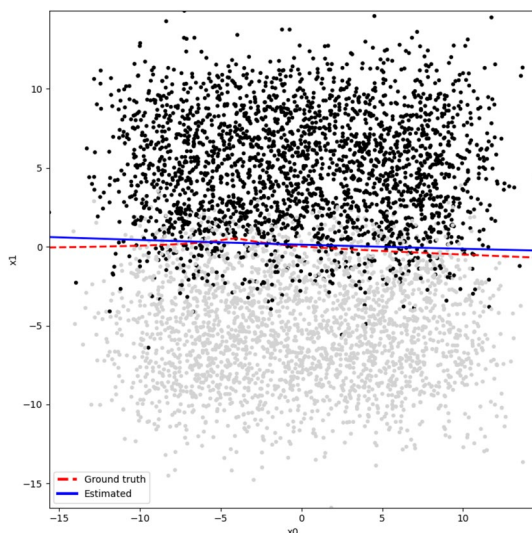


Fig. 4 This figure shows the target decision boundary (dashed red line), and the decision boundary found by the EDC algorithm (solid blue line). The target equation is sampled from a richer grammar than available to EDC. The target achieves a lower AUC (0.959) compared to the EDC algorithm (0.967)

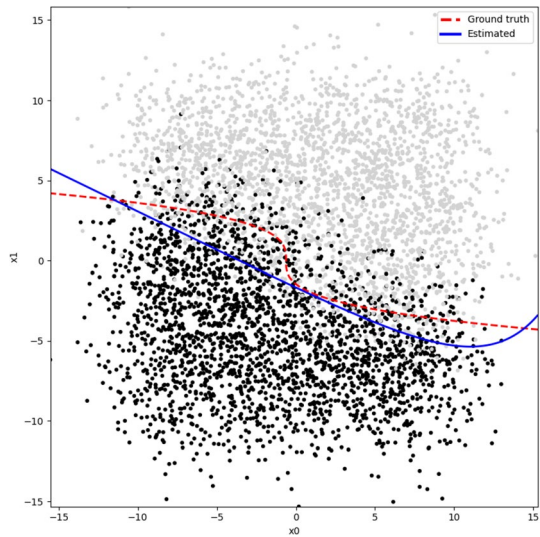
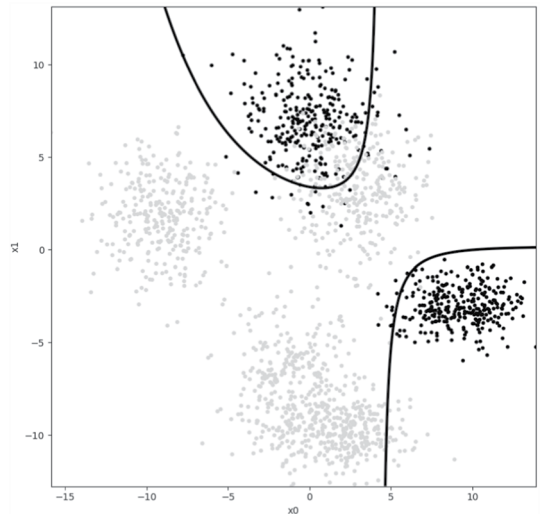


Fig. 5 The proposed decision boundary for the Gaussian clusters artificial dataset. Note that the discovered equation indicated above the figure is produced after translating the equation back to the non-normalised space. This introduces two additional constants for each feature x_i



consists of an interaction term between x_0 and x_1 . This term introduces an asymptote at $x_0 \approx 4.258$ and $x_1 \approx 0.125$ (for positive values of x_0), obtained by solving $f(x) = 0$. The last term $-0.66 \cdot \exp(-0.28x_0)$ pulls up the decision boundary for negative values of x_0 .

Table 2 shows the results of the comparison between the different classifiers. The state-of-the-art classification algorithms perform the best, with mean AUC values around 0.97. We show that the EDC algorithm outperforms the other SR-based classification approaches, but remains slightly behind the blackbox methods in terms of AUC. Furthermore, we observe that the explainable methods represent the section of the ranking, with a notably high standard deviation across the 100 generated datasets. We hypothesise that this is due to the strong dependence on the class being linearly separable, which will typically not be the case in the automatically generated datasets. For the performance of our proposed EDC

Table 2 This table shows the results of the artificial Gaussian clusters experiment

The results are presented in terms of mean AUC with the standard deviation in parentheses. The proposed EDC algorithm performs comparable to the state-of-the-art classification and outperforms all other explainable methods

Algorithm	AUC
MLP	0.972 (± 0.034)
SVM	0.970 (± 0.038)
RF	0.967 (± 0.037)
EDC	0.965 (± 0.043)
PySR	0.954 (± 0.052)
EGGP	0.932 (± 0.079)
Tree	0.899 (± 0.079)
M4GP	0.894 (± 0.082)
LDA	0.811 (± 0.139)
AMAXSC	0.802 (± 0.181)

Table 3 Overview of UCI dataset details

Dataset	Instances	#Numeric	#Nominal	#Pre-proc
ADULT (Becker and Kohavi 1996)	48 842	6	7	49
BANKNOTE (Lohweg 2013)	1372	4	0	0
BREAST (Zwitter and Soklic 1988)	286	0	10	39
CREDIT (Quinlan 1987)	690	6	9	35
CYLINDER (Evans 1994)	378	20	19	77
DIABETES	768	8	0	0
IONOSPHERE (Sigillito et al. 1989)	351	35	0	0
OCCUPANCY (Candanedo 2016)	20 560	5	0	0
SONAR (Sejnowski and Gorman 1988)	208	60	0	0

Attribute counts exclude class attributes. All datasets concern binary classification. The #Pre-proc. column indicates the number of attributes resulting from the one-hot encoding of nominal attributes and filtered on $p > 0.02$

method, we note that it performs on par with the state of the art and outperforms the existing explainable methods.

5.3 Real-World Datasets

The final set of experiments in this paper involves the evaluation of the EDC algorithm on real-world datasets. A list of UCI datasets that involve binary classification is selected. Table 3 shows an overview of the statistics of the datasets selected in this work. For comparison to EDC, we again apply the earlier list of nine classifiers.

Table 4 shows the results of the different classifiers in terms of average AUC on the test set for 10-fold cross-validation. When comparing the results of the EDC algorithm to the results of the other ED-based classifiers, we observe that for all datasets, the EDC algorithm achieves a higher AUC. This shows that our approach outperforms the current state-of-the-art ED-based approaches on binary classification. Furthermore, as noted in the introduction, the produced model, which is a single equation, is more interpretable than the results obtained from the M4GP algorithm. When comparing the results to the other interpretable classification algorithms, we note that LDA achieves very similar scores. We note that our EDC algorithm outperforms the simple decision tree algorithm in all datasets. When comparing the results to the state-of-the-art classification methods, we observe that the random

Table 4 Results of the classifiers on the UCI datasets

Dataset	EDC	AMAXSC	M4GP	PySR	EGGP
ADULT	0.897 (± 0.00)	0.806 (± 0.00)	0.720 (± 0.01)	0.867 (± 0.01)	0.868 (± 0.01)
BANKNOTE	1.000 (± 0.00)	0.988 (± 0.01)	0.982 (± 0.05)	1.000 (± 0.00)	1.000 (± 0.00)
BREAST	0.662 (± 0.12)	0.641 (± 0.12)	0.645 (± 0.11)	0.647 (± 0.13)	0.657 (± 0.07)
CREDIT	0.930 (± 0.03)	0.910 (± 0.04)	0.864 (± 0.05)	0.907 (± 0.04)	0.912 (± 0.04)
CYLINDER	0.728 (± 0.10)	0.575 (± 0.06)	0.662 (± 0.09)	0.674 (± 0.07)	0.686 (± 0.10)
DIABETES	0.831 (± 0.04)	0.803 (± 0.05)	0.715 (± 0.04)	0.813 (± 0.04)	0.821 (± 0.05)
IONOSPHERE	0.910 (± 0.07)	0.804 (± 0.09)	0.830 (± 0.08)	0.876 (± 0.08)	0.866 (± 0.08)
OCCUPANCY	0.996 (± 0.00)	0.994 (± 0.00)	0.850 (± 0.04)	0.994 (± 0.00)	0.993 (± 0.00)
SONAR	0.871 (± 0.07)	0.751 (± 0.06)	0.733 (± 0.05)	0.822 (± 0.08)	0.782 (± 0.08)
Average Score	0.869	0.808	0.778	0.845	0.843
Average Rank	3.56	8.11	9.11	6.44	6.11
Dataset	MLP	RF	SVM	LDA	Tree
ADULT	0.902 (± 0.00)	0.880 (± 0.00)	0.898 (± 0.00)	0.902 (± 0.00)	0.730 (± 0.01)
BANKNOTE	1.000 (± 0.00)	1.000 (± 0.00)	1.000 (± 0.00)	1.000 (± 0.00)	0.985 (± 0.01)
BREAST	0.712 (± 0.10)	0.684 (± 0.08)	0.720 (± 0.08)	0.665 (± 0.13)	0.613 (± 0.09)
CREDIT	0.907 (± 0.04)	0.927 (± 0.03)	0.913 (± 0.03)	0.923 (± 0.04)	0.810 (± 0.05)
CYLINDER	0.822 (± 0.07)	0.860 (± 0.08)	0.773 (± 0.07)	0.762 (± 0.08)	0.652 (± 0.09)
DIABETES	0.837 (± 0.04)	0.820 (± 0.04)	0.834 (± 0.03)	0.830 (± 0.03)	0.688 (± 0.04)
IONOSPHERE	0.981 (± 0.02)	0.983 (± 0.01)	0.982 (± 0.02)	0.900 (± 0.06)	0.890 (± 0.05)
OCCUPANCY	0.997 (± 0.00)	0.999 (± 0.00)	0.993 (± 0.00)	0.994 (± 0.00)	0.989 (± 0.00)
SONAR	0.926 (± 0.06)	0.930 (± 0.04)	0.935 (± 0.04)	0.819 (± 0.08)	0.716 (± 0.08)
Average Score	0.898	0.898	0.894	0.866	0.786
Average Rank	2.67	2.89	2.67	4.22	9.11

All scores are the mean area under the receiver operator curve (AUC) across 10 folds, with the standard deviation reported in parentheses

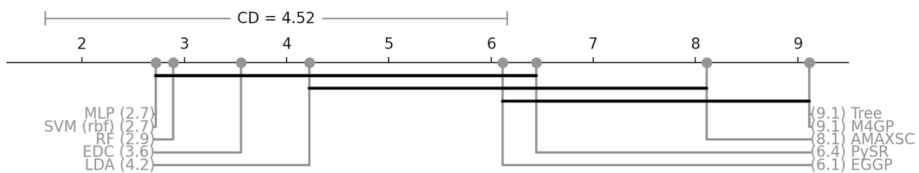


Fig. 6 Critical distance plot of the ranks for the different classifiers for the UCI datasets. The top bar shows the critical distance (CD), which in our setup equals 4.52. EDC outperforms EGGP, PySR, AMAXSC, M4GP, and the decision tree, although not statistically significant. Similarly, MLP, RF, and SVM perform better but not statistically significantly

forest method outperforms our approach for 4 of the 9 datasets. Furthermore, for the datasets IONOSPHERE and SONAR, the random forest, MLP, and SVM algorithms show substantially higher AUC scores compared to EDC. This would indicate a relationship between the input features and class that is not present in our building blocks.

Figure 6 shows a critical distance plot, as described by Demšar (2006), of the ranks for the different classifiers evaluated in this work. The following equation obtains the value for the critical distance

$$CD = q_\alpha \sqrt{\frac{k(k+1)}{6N}}$$

where k denotes the number of classifiers used in the comparison, which in our case is 10, and N denotes the number of datasets used in the comparison (9). The critical value q_α for $\alpha = 0.05$ and 10 classifiers equals 3.164. Putting this together, we obtain $CD = 4.52$.

We identify three overlapping groups of classifiers: the first group consists of the MLP, RF, SVM, EDC, LDA, PySR, and EGGP algorithms. The second group contains the LDA, EGGP, PySR, and AMAXSC algorithms. The third and final group contains the EGGP, PySR, AMAXSC, M4GP and decision tree algorithms. This plot shows that the current state-of-the-art classifiers outperform the decision tree, M4GP, and AMAXSC algorithms. However, current state-of-the-art classification algorithms do not significantly outperform our proposed EDC algorithms. Finally we observe that the EDC is closer to the black box algorithms in terms of performance when compared to the PySR, and EGGP, algorithms, which in turn outperform the other interpretable binary classification methods

Equation 3 shows the obtained equation for the ADULT dataset, as a demonstration of the interpretable nature of EDC's output. The task in the ADULT dataset is to classify whether a person earns more than \$50,000 a year. We observe some interesting phenomena in the obtained equation.

$$0.75 - 1.27 \cdot \text{own-child} \cdot \text{education-num} + 3.37 \cdot \text{capitalgain} + 8.01 \cdot \exp(8.18 \cdot \text{married-civ-spouse}) \quad (3)$$

First, we show that the summand $-1.27 \cdot \text{own-child} \cdot \text{education-num}$ effectively operates as an *if* statement. If the person is a child in the household ($\text{own-child} = 1$), the education-num (years of education received) is counted as a negative penalty with scaling factor 1.27 (making *true* less likely). Second, the numeric value capital-gain is a positive influence with coefficient 3.37. Finally, the summand $8.01 \cdot \exp(8.18 \cdot \text{married-civ-spouse})$, while appearing quite complex, effectively operates as an *if-else* statement. If the person is married, living with their spouse, add $8.01 \cdot e^{8.18} \approx 28586$, else only add $8.01 \cdot e^0 = 8.01$.

You could argue that Eq. 3 is an overly convoluted expression, essentially describing linear effects and two *if* statements. A more interpretable model can be obtained by post-processing the equations, searching for implicit *if* statements wherever binary factors are involved. Alternatively, the core grammar could be upgraded to allow simple conditionals explicitly, e.g. if X then c else c . This would marginally increase the run time, but otherwise wouldn't affect the expressive power of the grammar.

5.3.1 Search Extent

EDC's standard setting assumes a search depth of $d = 6$, which, assuming all summands can involve one or two features, produces equations of at most 12 features. When confronted with a dataset whose correct classification requires the involvement of 13 or more features, EDC would not be able to produce an optimal classifier, whereas a more basic algorithm like LDA does not have this limitation. However, the experiments in the previous section (for example, see Fig. 6) suggest that EDC's maximum equation complexity is not a limitation in real-world datasets (for example, because EDC's average rank (3.6) is bet-

ter than LDA's (4.2)). Still, EDC's search is fairly extensive and computationally costly, so it's a fair question whether less extensive search could perhaps achieve comparable results.

In this section, we compare the normal EDC setting with three computationally cheaper settings, and analyse the trade-off between computation and classification accuracy. The first setting, *Shallow*, simply examines EDC with a maximum search depth of $d = 3$ which implies a maximum number of features of 6. Considering how beam search works, this should result in about half of the candidate equations considered (and potentially a drop in accuracy).

In a second setting (*Simple*), we extend the setting of *Shallow* by reducing the expressive power of the grammar G . One of the biggest determinants of computation time in EDC comes from the interaction terms $X \cdot X$. While being potentially powerful in terms of generating non-linear decision boundaries (e.g., Fig. 5), the interaction terms come at a considerable cost, because the number of summands grows quadratically in the number of features. Instead of the interaction terms, we just add a quadratic term $c \cdot X^2$, for which there are only a linear number of summands. This results in the following simplified grammar:

- $\mathcal{N}_a = \{V, B, X\}$
- $\mathcal{T}_a = \{x_1, c, +, \cdot, \exp\}$
- $\mathcal{R}_a = \{V \rightarrow c \mid V + B, B \rightarrow c \cdot X \mid c \cdot \exp(c \cdot X) \mid c \cdot X^2, X \rightarrow x_1\}$
- $\mathcal{S}_a = \{V\}$.

In the final setting (*Narrow*), we keep the original search depth ($d = 6$) and grammar, but set the beam width to $w = 2$. This forces the algorithm to prune seemingly uninteresting branches earlier in the search process. This has the downside that some patterns seem interesting only at greater search depths, e.g. with the XOR problem.

Table 5 reports on the details of this comparison. Judging by the Average score and Average rank, the differences between the four settings are minimal, suggesting that the search extent of the *Normal* setting can be reduced in datasets where complex interactions and

Table 5 Results of the different configurations of our EDC algorithm on the UCI datasets

Dataset	Normal	Shallow	Simple	Narrow
ADULT	0.897 (± 0.00)	0.889 (± 0.00)	0.880 (± 0.00)	0.880 (± 0.00)
BANKNOTE	1.000 (± 0.00)	1.000 (± 0.00)	1.000 (± 0.00)	1.000 (± 0.00)
BREAST	0.662 (± 0.12)	0.670 (± 0.12)	0.697 (± 0.08)	0.691 (± 0.07)
CREDIT	0.930 (± 0.03)	0.918 (± 0.03)	0.924 (± 0.04)	0.922 (± 0.04)
CYLINDER	0.728 (± 0.10)	0.735 (± 0.10)	0.640 (± 0.09)	0.680 (± 0.09)
DIABETES	0.831 (± 0.04)	0.830 (± 0.04)	0.833 (± 0.03)	0.833 (± 0.03)
IONOSPHERE	0.910 (± 0.07)	0.894 (± 0.06)	0.861 (± 0.07)	0.892 (± 0.07)
OCCUPANCY	0.996 (± 0.00)	0.996 (± 0.00)	0.996 (± 0.00)	0.996 (± 0.00)
SONAR	0.871 (± 0.07)	0.780 (± 0.11)	0.827 (± 0.08)	0.819 (± 0.11)
Average AUC	0.869	0.857	0.851	0.857
Average Rank	1.9	2.4	2.7	2.9
#Candidates (SONAR)	189,720	78,120	3,780	37,940
#Candidates (ADULT)	127,449	52,479	3,087	25,489

In total, four different configurations have been tested. *Normal* refers to the regular setting at $d = 6$ with interaction terms. *Shallow* is like *Normal*, but with $d = 3$. *Simple* refers to the reduced grammar *without* interaction terms (but with squared terms) at $d = 3$. *Narrow* is the *Normal* setting with a beam width of 2 instead of 10

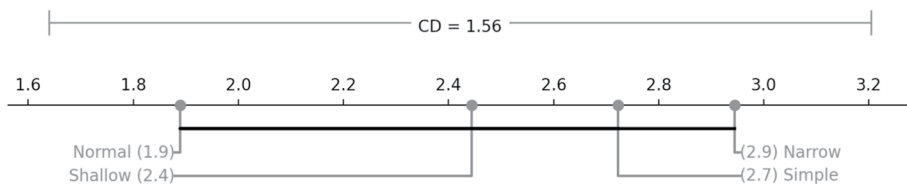


Fig. 7 Critical distance plot of the ranks of the different grammars for the EDC algorithm on the UCI datasets

many (strongly) relevant features are not expected. Figure 7 shows the critical distance plot of the results of the different grammar configurations. Overall, across all datasets analysed, there was no significant difference in AUC scores, although the overall performance of the three alternatives is slightly lower than the original proposed setting.

Trading in accuracy for speed, the *Shallow*, *Simple*, and *Narrow* settings are substantially faster than the *Normal* setting. The bottom rows of Table 5 give two examples of how the reduced depth and grammatical expressivity affect the number of candidates considered and thus the number of optimisation steps. The overall run time is governed by the inner loop of the optimisation, and the search process plays a negligible role.

As expected, the *Shallow* setting is slightly more than twice as fast as the *Normal* setting. Since the first search level involves relatively fewer candidates than subsequent levels, the factor is not exactly 2. However, a bigger gain can be achieved with the reduced grammar of the *Simple* setting, which doesn't involve the expensive interaction terms. Compared to the *Normal* setting, this produces a speed-up of a factor 40 to 50, depending of course on the number of features in the specific dataset. *Narrow* is comparable in number of candidates and average AUC, but still substantially slower than *Simple*. To conclude, Table 5 shows a trade-off that can be exploited in cases where compute time is a priority. The results demonstrate that substantial speed-ups can be achieved at only a minimal reduction in classification accuracy. At the same time, when marginal gains are a priority, more extensive computation can lead to competitive results.

6 Discussion & Conclusion

We have proposed a new classification algorithm based on equation discovery and thoroughly tested the method on artificial and real-world data (from the UCI Repository). Starting with the challenge of finding the optimal parameters for a given candidate equation and dataset, we compare different flavours of optimisation approaches, ranging from undirected random search to off-the-shelf optimisers. It turns out that, within our suggested grammar of modest size, our EDC-specific optimisation procedure outperforms more generic optimisation procedures, including gradient descent.

Having established the adequate performance of the optimisation step, we moved on to test whether the heuristic search for equations in EDC was able to discover the correct equation and, thus, a successful decision boundary. On various artificial datasets where the target equation is known, this was indeed the case. In experiments where the target equation comes from the same grammar as EDC uses, AUC scores on datasets free of noise were in the order of 0.1% from the perfect score. Further experiments with artificial data

that includes modest to substantial noise showed an interesting phenomenon, where EDC outperformed the score of the target equation. We deliberately avoid the term ‘ground truth’, since apparently, as soon as you introduce noise (data noise, not label noise), the implicit decision boundary embedded in the data may shift slightly from the original boundary. For linear decision boundaries, this is very limited and only due to the random sample. However, in curved decision boundaries, it appears that the implicit boundary shifts towards the ‘enclosed’ region. The experiments demonstrate that the higher the degree of noise in a dataset, the simpler its decision boundary becomes, the less complex a grammar has to be. This offers opportunities for faster EDC runs, for example by reducing the search depth or leaving out interaction terms.

It should be noted that the discovered equations, despite scoring well, may not necessarily take on the same structure as the original. There apparently is some redundancy in our selected pattern language, such that different equations can still produce roughly the same decision boundary. Random effects in the sampling of the dataset or in the Gaussian noise allow some fuzziness in the boundary, which EDC sometimes exploits. For this reason, we have not included a grammatical validation between equations.

The final test concerned experiments on real-world data, for which typically no explicit decision boundary is known. The results indicate that EDC is able to compete with state-of-the-art methods, such as MLPs and RFs, and outperforms all other ED-based classification methods. EDC is only marginally better (average rank 3.6 vs. 4.2) than LDA, and occasionally, LDA wins (ADULT, BREAST, CYLINDER), and we hypothesise that this is due to a larger number of relevant features required. After all, EDC should be able to discover the same linear equations as LDA uses to discriminate between the classes, except for the upper limit on the number of relevant features for EDC (double the search depth, i.e., 12 relevant features). Apparently, in rare cases, high numbers of relevant features do occur in real-world dataset, but the scores in Table 4 demonstrate that this is not frequently the case. Most classification tasks apparently can be solved with relatively few features.

Finally, we must address the run time of the EDC algorithm. As stated, this is significantly longer than the algorithms used in our comparison, which have run times of < 1 second. The cause of this is the high number of function evaluations required in the search. Especially the building block $x_i \cdot x_j$ introduces a quadratic number of building blocks. This, combined with the feature expansion from one-hot encoding categorical features, results in long run times for the EDC algorithm in its normal, ‘luxurious’ setting. We should note that our selected hyper-parameters were not optimised for run time, so they are likely on the excessive side. Furthermore, we note that the optimisation budget, 1000 in our case, is excessive for some scenarios. A dynamic allocation of optimisation steps could improve runtime without degrading the algorithm’s overall performance. Our experiments in Sect. 5.3.1 demonstrate that considerable further performance gains can be obtained by reducing the search extent in terms of search depth, expressivity of the grammar, and beam width. Substantially less extensive settings than the standard setting can offer speed-ups up to a factor 50.

Although our algorithm is geared towards binary classification, many of the standard approaches for multi-class classification using binary classifiers can be applied to EDC if the class variable is not binary (Bishop, 2006). We find particularly the *one-vs-rest* approach to be quite applicable to our ambition of interpretable classification, as each of the k classifiers (k being the number of classes) explains under what circumstances a particular class

label prevails. Furthermore, a one-vs-rest policy is known to compete with or outperform alternative multi-class classification algorithms (Rifkin & Klautau, 2004).

Author Contributions G. and A. wrote the main manuscript text. G. prepared the figures and tables. All Authors reviewed the manuscript.

Data Availability The data is available online in the url present in the manuscript.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by/4.0/>.

References

- Antonov, K., Kalkreuth, R., Yang, K., Bäck, T., Stein, N., & Kononova, A. (2024). A functional analysis approach to symbolic regression. In: *Proceedings of the genetic and evolutionary computation conference* (pp. 859–867). ACM. <https://doi.org/10.1145/3638529.3654079>
- Augusto, D. A., & Barbosa, H. J. (2000). Symbolic regression via genetic programming. *Proceedings sixth brazilian symposium on neural networks* (pp. 173–178)
- Becker, B., & Kohavi, R. (1996). Adult. *UCI Machine Learning Repository*. <https://doi.org/10.24432/C5XW20>
- Billard, L., & Diday, E. (2002). Symbolic regression analysis. *Classification, clustering, and data analysis: Recent advances and applications* (pp. 281–288)
- Bishop, C. M. (2006). *Pattern recognition and machine learning*. Information Science and Statistics. Springer.
- Boyd, S., & Vandenberghe, L. (2004). *Convex optimization*. Cambridge University Press.
- Candanedo, L. (2016). Occupancy detection. *UCI Machine Learning Repository*. <https://doi.org/10.24432/C5X01N>
- Cava, W.L., Singh, T.R., Taggart, J., Suri, S., & Moore, J.H. (2019). Learning concise representations for regression by evolving networks of trees. <https://arxiv.org/abs/1807.00981>
- Cranmer, M. (2023). Interpretable machine learning for science with PySR and SymbolicRegression.jl. <https://arxiv.org/abs/2305.01582>
- Cybenko, G., O'Leary, D. P., & Rissanen, J. (1999). *The mathematics of information coding*. Extraction and Distribution. Springer.
- Demšar, J. (2006). Statistical comparisons of classifiers over multiple data sets. *Journal of Machine Learning Research*, 7, 1–30.
- Evans, B. (1994). Cylinder bands. *UCI Machine Learning Repository*. <https://doi.org/10.24432/C50C7B>
- Flach, P. (2012). *Machine learning: The art and science of algorithms that make sense of data*. Cambridge University Press.
- Haeri, M. A., Ebadzadeh, M. M., & Folino, G. (2017). Statistical genetic programming for symbolic regression. *Applied Soft Computing*, 60, 447–469.
- Hosmer, D. W., & Lemeshow, S. (2000). *Applied logistic regression*. Wiley.
- Ingalalli, V., Silva, S., Castelli, M., & Vanneschi, L. (2014). A multi-dimensional genetic programming approach for multi-class classification problems. In: *Genetic programming* (pp. 48–60). Springer.
- Jiang, N., Wang, Z., & Xue, Y. (2026). EGG-SR: Embedding symbolic equivalence into symbolic regression via equality graph. <https://arxiv.org/abs/2511.05849>
- Korns, M.F. (2018). In: Riolo, R., Worzel, B., Goldman, B., Tozier, B. (eds.) *An Evolutionary algorithm for big data multi-class classification problems* (pp. 165–178). Springer. https://doi.org/10.1007/978-3-319-97088-2_11
- Kumar, A., Vembu, S., Menon, A. K., & Elkan, C. (2013). Beam search algorithms for multilabel learning. *Machine Learning*, 92, 65–89.

- La Cava, W., Silva, S., Danai, K., Spector, L., Vanneschi, L., & Moore, J.H. (2019). Multidimensional genetic programming for multiclass classification. *Swarm and Evolutionary Computation* 44, 260–272. <https://doi.org/10.1016/j.swevo.2018.03.015>
- Levine, I. (1978). *Physical chemistry*. McGraw-Hill.
- Lindauer, M., Eggensperger, K., Feurer, M., Biedenkapp, A., Deng, D., Benjamins, C., Ruhkopf, T., Sass, R., & Hutter, F. (2022). SMAC3: A versatile Bayesian optimization package for hyperparameter optimization. *Journal of Machine Learning Research*, 23(54), 1–9.
- Lohweg, V. (2013). Banknote authentication. *UCI Machine Learning Repository*. <https://doi.org/10.24432/C55P57>
- Makke, N., & Chawla, S. (2024). Interpretable scientific discovery with symbolic regression: A review. *Artificial Intelligence Review*. <https://doi.org/10.1007/s10462-023-10622-0>
- Meeng, M., Duivesteijn, W., & Knobbe, A. (2014). ROCsearch - an ROC-guided search strategy for subgroup discovery. *Proceedings of the 2014 SIAM International Conference on Data Mining* (pp. 704–712). SIAM.
- Muñoz, L., Silva, S., & Trujillo, L. (2015). M3GP - multiclass classification with GP. *European Conference on Genetic Programming* (pp. 78–91). Springer.
- Quinlan, J. R. (1987). Credit approval. *UCI Machine Learning Repository*. <https://doi.org/10.24432/C5FS30>
- Rifkin, R., & Klautau, A. (2004). In defense of one-vs-all classification. *Journal of Machine Learning Research*, 5, 101–141.
- Schwab, I., & Link, N. (2011). Reusable knowledge from symbolic regression classification. In: *2011 Fifth international conference on genetic and evolutionary computing*, (pp. 106–109). <https://doi.org/10.1109/ICGEC.2011.34>
- Sejnowski, T., & Gorman, R. (1988). Connectionist bench (Sonar, Mines vs. Rocks). *UCI Machine Learning Repository*. <https://doi.org/10.24432/C5T01Q>
- Sigillito, V., Wing, S., Hutton, L., & Baker, K. (1989). Ionosphere. *UCI Machine Learning Repository*. <https://doi.org/10.24432/C5W01B>
- Todorovski, L., & Dzeroski, S. (1997). Declarative bias in equation discovery. *Proceedings of the fourteenth international conference on machine learning. ICML '97* (pp. 376–384). Morgan Kaufmann Publishers Inc.
- Toussaint, G., & Knobbe, A. (2025). EDC: Equation discovery for classification. In: *Proceedings discovery science 2025*, (pp. 128–142). Springer. https://doi.org/10.1007/978-3-032-05461-6_9
- Zwitter, M., & Soklic, M. (1988). Breast cancer. *UCI Machine Learning Repository*. <https://doi.org/10.24432/C51P4M>

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.